

개정3판

Visual
Studio
2017

쉽게 풀어쓴

C언어 EXPRESS



천인국 지음

CHAPTER 2: 프로그램 작성 과정



프로그램 개발 과정

요구사항 분석

무엇을
만들 것인가를
결정한다.



설계

알고리즘을
설계한다.



구현

개발 도구를
사용하여
소스 코드를
작성한다.



테스팅

여러가지
경우에 대하여
실행하여 본다.



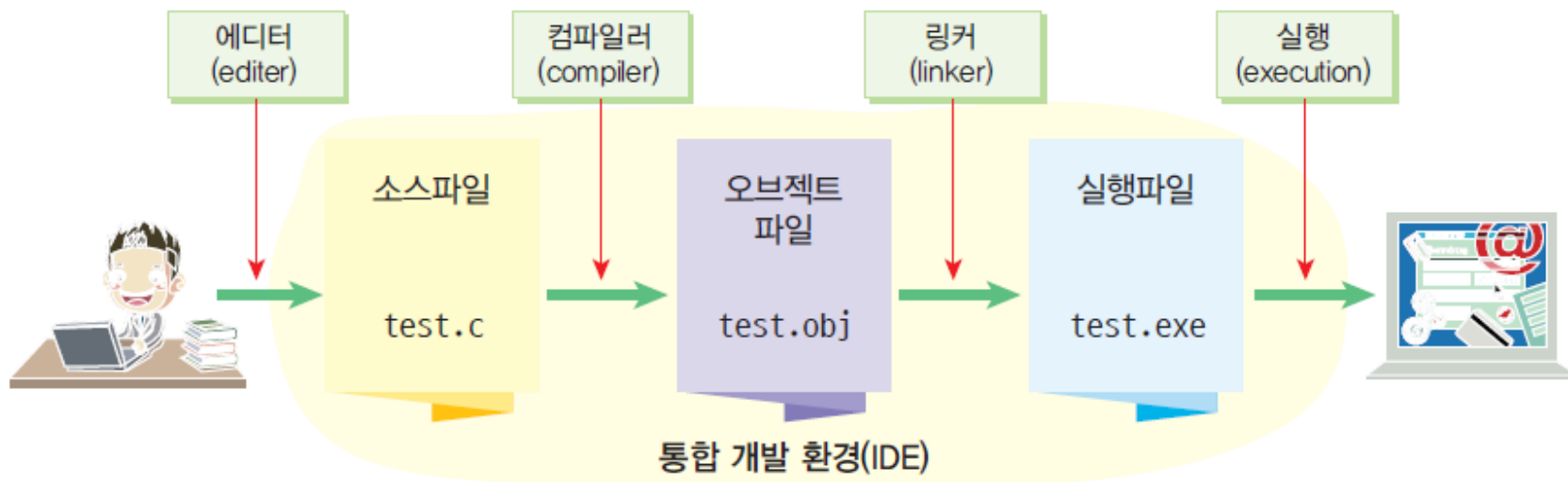
유지보수

사용자의 추가
요구사항을
반영한다.





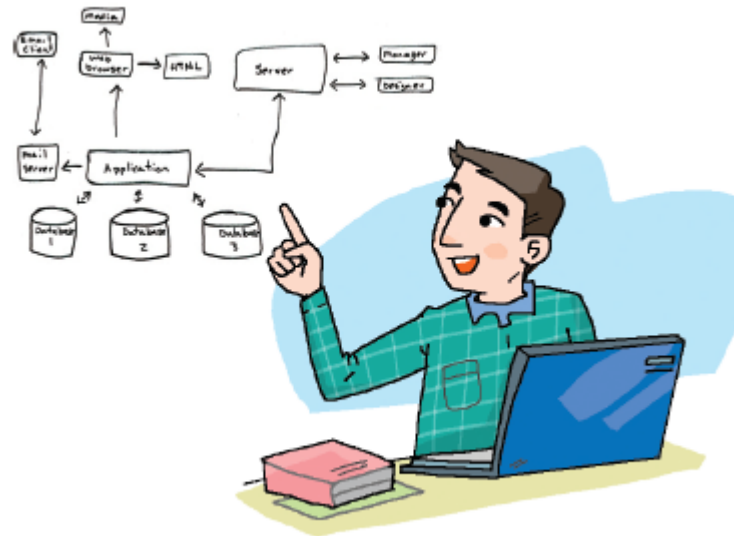
프로그램 개발 과정





설계

- 문제를 해결하는 알고리즘을 개발하는 단계
- 순서도와 의사 코드를 도구로 사용
- 알고리즘은 프로그래밍 언어와는 무관
- 알고리즘은 원하는 결과를 얻기 위하여 밟아야 하는 단계에 집중적으로 초점을 맞추는 것





소스 작성

- 알고리즘의 각 단계를 프로그래밍 언어를 이용하여 기술
- 알고리즘을 프로그래밍 언어의 문법에 맞추어 기술한 것을 *소스 프로그램(source program)*
- 소스 프로그램은 주로 텍스트 에디터나 통합 개발 환경을 이용하여 작성
- 소스 파일 이름: (예) **test.c**

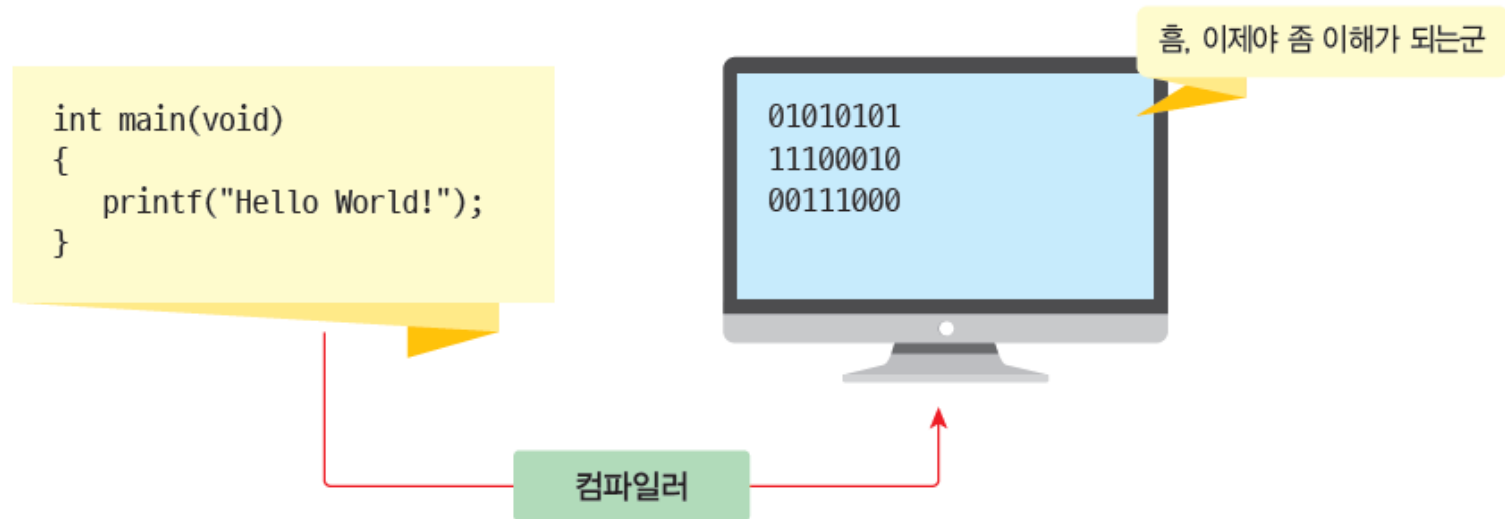


```
int main(void)
{
    printf("Hello World!");
}
```



컴파일

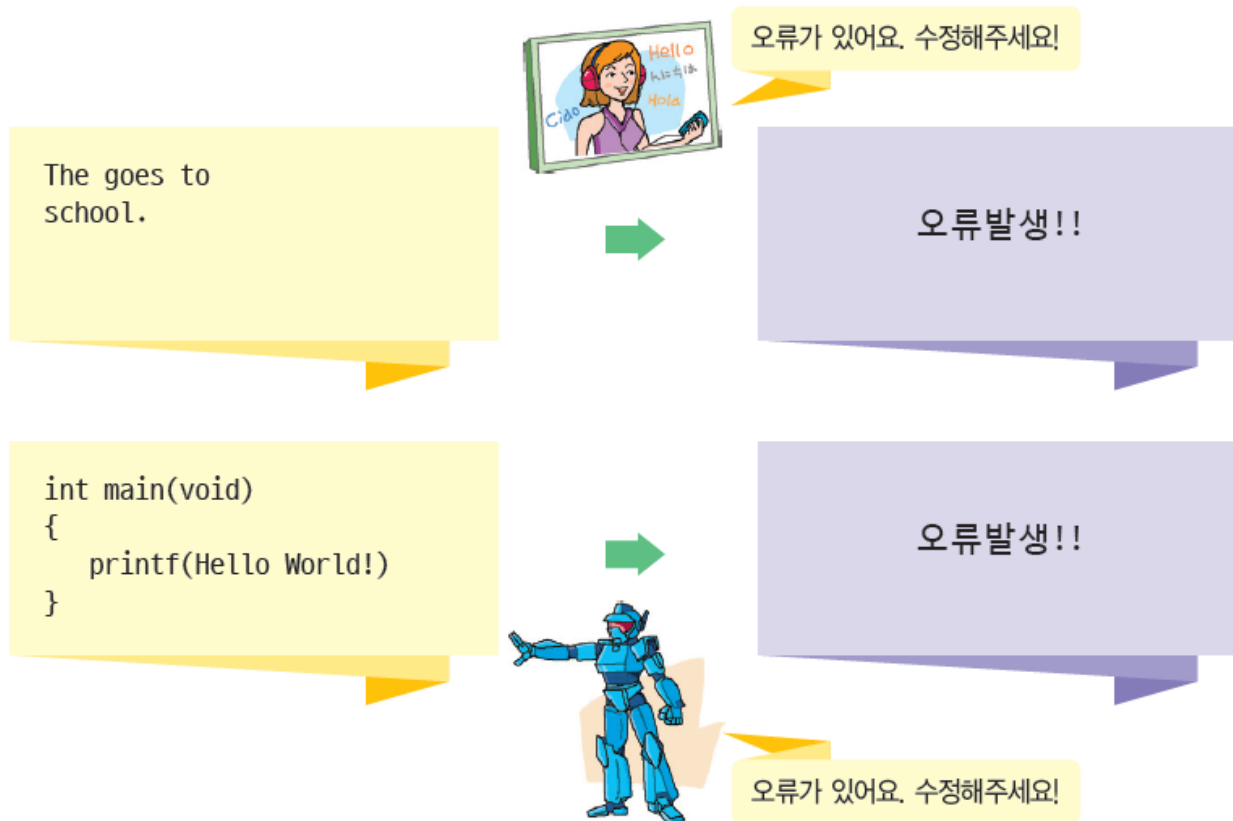
- 소스 프로그램을 오브젝트 파일로 변환하는 작업
- 오브젝트 파일 이름: (예) test.obj





컴파일 오류

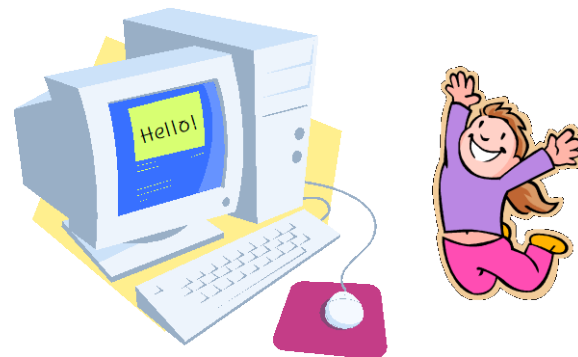
- 컴파일 오류(compile error): 문법 오류
 - ▣ (예) He go to school;





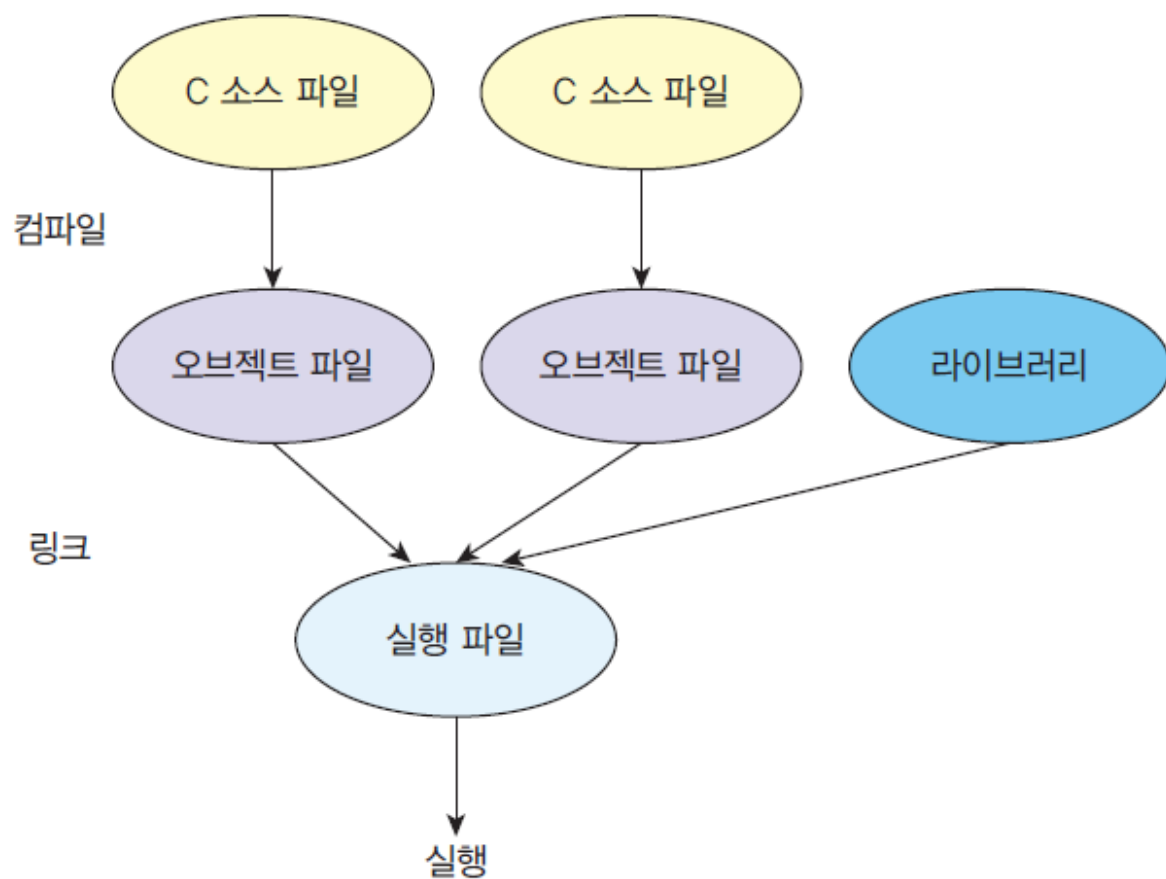
링크

- 컴파일된 목적 프로그램을 라이브러리와 연결하여 실행 프로그램을 작성하는 것
- 실행 파일 이름: (예) **test.exe**
- *라이브러리(library)*: 프로그래머들이 많이 사용되는 기능을 미리 작성해 놓은 것
 - ▣ (예) 입출력 기능, 파일 처리, 수학 함수 계산
- 링크를 수행하는 프로그램을 *링커(linker)*라고 한다.





링크

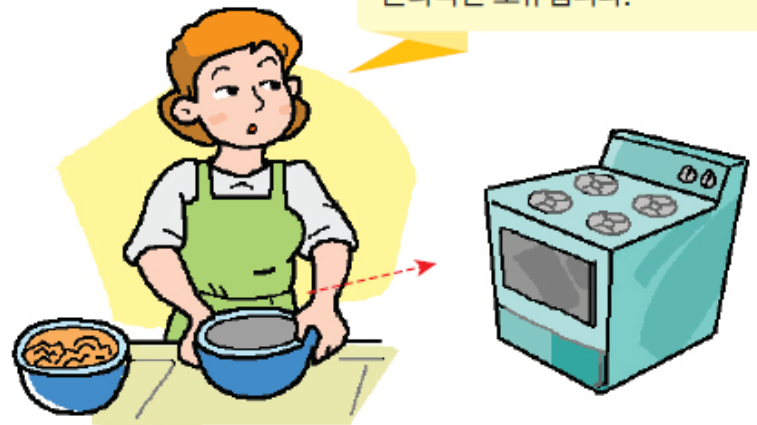




실행 및 디버깅

- 실행 시간 오류(run time error):
 - ▣ (예) 0으로 나누는 것
 - ▣ 잘못된 메모리 주소에 접근하는 것
- 논리 오류(logical error):
 - ▣ 문법은 틀리지 않았으나 논리적으로 정확하지 않는 것
 - ▣ ① 그릇1과 그릇2를 준비한다.
② 그릇1에 밀가루, 우유, 계란을 넣고 잘 섞는다.
③ 그릇2를 오븐에 넣고 30분 동안 350도로 굽는다.

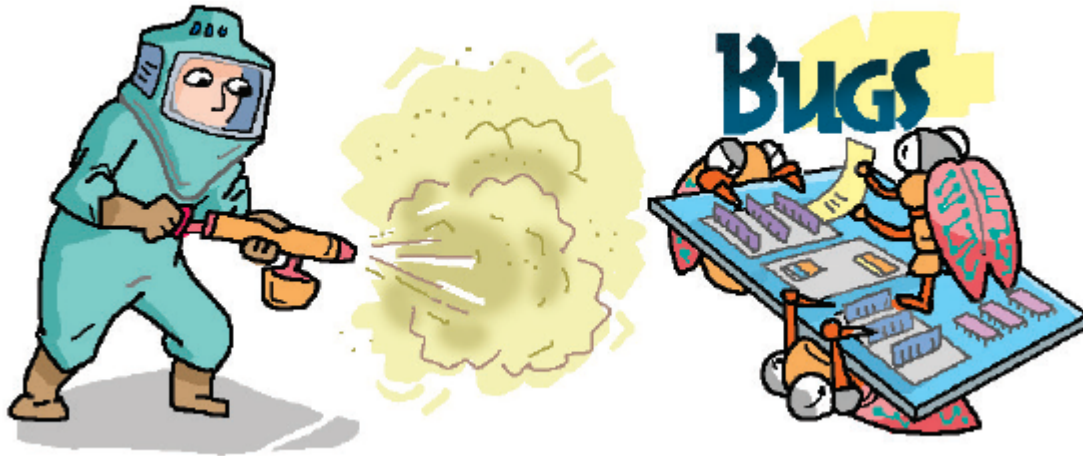
실수로 빈그릇을 오븐에 넣는다면
논리적인 오류입니다.





디버깅

- 소스에 존재하는 오류를 잡는 것





디버깅의 유래

- 1945년 마크 II 컴퓨터가 릴레이 장치에 날아든 나방 때문에 고장을 일으켰고 이것을 “컴퓨터 버그(bug: 벌레)”
- 라고 불렀다. 여성 컴퓨터 과학자인 그레이스 호퍼가 나방을 채집해 기록에 남기고 이를 “디버깅(debugging)”작업이라고 보고하였다



9/2
9/9

0800 Machine started
1000 stopped - on fire ✓ { 1200 9.030 447.045
1300 (1300) HP - MC 2.130476955 9.037 896.595 count
2.130476955 (5.000) 9.012 925.059 (-1)
2.130476955
2.130476955
2.130476955
2.130476955
Relays 6-2 in 0.24 fault speed speed test
1m relay 11.00 test.

1100 Started Cosine Taps (Sine check)
1525 Started Multiplier Test.

1545 Relay 40 Panel F (motion relay).

1700 First actual case of bug being found.
1730 Machine started.
1700 closed down.



소프트웨어의 유지 보수

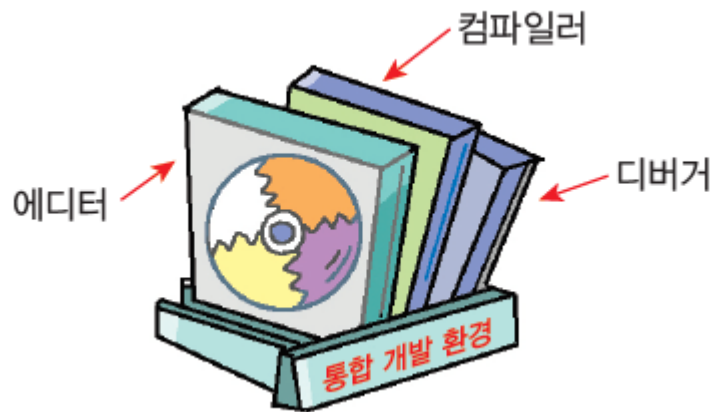
- 소프트웨어의 유지 보수가 필요한 이유
 - ▣ 디버깅 후에도 버그가 남아 있을 수 있기 때문
 - ▣ 소프트웨어가 개발된 다음에 사용자의 요구가 추가될 수 있기 때문
- 유지 보수 비용이 전체 비용의 **50%** 이상을 차지





통합 개발 환경

- 통합 개발 환경 (IDE: integrated development environment)
 - ▣ 에디터 + 컴파일러 + 디버거





통합 개발 환경의 예

- 비주얼 스튜디오: 마이크로소프트
- 이클립스(eclipse): 오픈 소스 프로젝트
- Dev-C++: 오픈 소스 프로젝트





비주얼 스튜디오 버전

- 커뮤니티(Visual Studio Community) 버전은 “기업 외 응용 프로그램 빌드 개발자를 위한 완벽한 기능의 확장 가능한 무료 도구”이다.
- 프로페셔널 버전(Visual Studio Professional)은 “개별 개발자 또는 소규모 팀을 위한 전문적인 개발자 도구 및 서비스”라고 되어 있다.
- 엔터프라이즈 버전(Visual Studio Enterprise)은 “고급 테스트 및 DevOps를 포함해서 어떠한 크기나 복잡한 프로젝트까지 개발 팀을 위한 고급 기능이 포함된 엔터프라이즈급 솔루션”라고 표시되어 있다.



비주얼 스튜디오 설치

The screenshot shows the Visual Studio Community website in Korean. The browser address bar displays 'visualstudio.microsoft.com/ko/vs/community/'. The page features the Microsoft logo and 'Visual Studio' branding. A prominent heading reads 'Visual Studio Community', followed by a description: 'Android, iOS 및 Windows용 최신 응용 프로그램뿐 아니라 웹 응용 프로그램 및 클라우드 서비스를 만들기 위한 모든 기능을 갖춘 확장 가능한 무료 IDE입니다.' (The latest applications for Android, iOS, and Windows, as well as web applications and cloud services, all in one free, extensible IDE). A purple button labeled 'Visual Studio 다운로드' (Download Visual Studio) is visible. Below this, a section titled '필요한 모든 것을 한 곳에서' (Everything you need in one place) lists four key features: '유연성' (Flexibility) for building all platform apps, '생산성' (Productivity) with design, editing, and debugging tools, '에코 시스템' (Ecosystem) with access to thousands of extensions, and '언어' (Language) support for C#, Visual Basic, F#, C++, HTML, JavaScript, TypeScript, and Python. A vertical '프리미엄' (Premium) badge is on the right side of the page.

무료 IDE 및 개발자 도구 | Visual x +

visualstudio.microsoft.com/ko/vs/community/

Microsoft | Visual Studio Visual Studio 2019 기능 자세히 무료 Visual Studio Microsoft 전체

Visual Studio Community

Android, iOS 및 Windows용 최신 응용 프로그램뿐 아니라 웹 응용 프로그램 및 클라우드 서비스를 만들기 위한 모든 기능을 갖춘 확장 가능한 무료 IDE입니다.

Visual Studio 다운로드

필요한 모든 것을 한 곳에서

- 유연성**
모든 플랫폼용 앱 빌드
- 생산성**
디자인, 편집기, 디버거, 프로파일러를 단일 도구로 제공
- 에코 시스템**
수천 개의 확장 액세스
- 언어**
C#, Visual Basic, F#, C++, HTML, JavaScript, TypeScript, Python 등으로 코딩

프리미엄



비주얼 스튜디오 설치

설치 중 — Visual Studio Community 2019 — 16.5.0

워크로드 개별 구성 요소 언어 팩 설치 위치

웹 및 클라우드 (4)

 ASP.NET 및 웹 개발
Docker 지원이 포함된 ASP.NET Core, ASP.NET, HTML/JavaScript 및 컨테이너를 사용하여 웹 애플리케이션...

 Python 개발
Python에 대한 편집, 디버깅, 대화형 개발 및 소스 제어입니다.

Azure 개발
.NET Core 및 .NET Framework를 사용하여 클라우드 앱을 개발하고 리소스를 만들기 위한 Azure SDK, 도구 및 프로젝트...

 Node.js 개발
비동기 이벤트 구동 JavaScript 런타임인 Node.js를 사용하여 확장 가능한 네트워크 애플리케이션을 빌드합니다.

데스크톱 및 모바일 (5)

 .NET 데스크톱 개발
.NET Core and .NET Framework와 함께 C#, Visual Basic 및 F#를 사용하여 WPF, Windows Forms 및 콘솔 애플리케이션...

 C++를 사용한 데스크톱 개발
MSVC, Clang, CMake 또는 MSBuild 등 선택한 도구를 사용하여 Windows용 최신 C++ 앱을 빌드합니다.

 유니버설 Windows 플랫폼 개발
C#, VB 또는 C++(선택 사항)를 사용하여 유니버설 Windows 플랫폼용 애플리케이션을 만듭니다.

 .NET을 사용한 모바일 개발
Xamarin을 사용하여 iOS, Android 또는 Windows용 플랫폼 간 애플리케이션을 빌드합니다.

위치

C:\Program Files (x86)\Microsoft Visual Studio\2019\Community 변경...

필요한 총 공간 7.07GB

다운로드하는 동안 설치 설치

설치 세부 정보

> Visual Studio 핵심 편집기

✓ C++를 사용한 데스크톱 개발
포함됨

- ✓ C++ 핵심 데스크톱 기능
- ✓ IntelliCode

옵션

- ✓ MSVC v142 - VS 2019 C++ x64/x86 빌드 도구(v...
- ✓ Windows 10 SDK(10.0.18362.0)
- ✓ Just-In-Time 디버거
- ✓ C++ 프로파일링 도구
- ✓ Windows용 C++ CMake 도구
- ✓ 최신 v142 빌드 도구용 C++ ATL(x86 및 x64)
- ✓ Test Adapter for Boost.Test
- ✓ Test Adapter for Google Test
- ✓ Live Share
- ✓ C++ AddressSanitizer(실험적)
- ☐ 최신 v142 빌드 도구용 C++ MFC(x86 및 x64)
- ☐ v142 빌드 도구용 C++/CLI 지원(14.25)
- ☐ v142 빌드 도구용 C++ 모듈(x64/x86 - 실험적)
- ☐ Windows용 C++ Clang 도구(9.0.0 - x64/x86)

계속하면 선택한 Visual Studio 버전

에 대한 라이선스에 동의하는 것입니다. Microsoft는 Visual Studio와 함께 다른 소프트웨어를 다운로드할 수 있는 기능도 제공합니다. 이 소프트웨어는 타사 고지 사항 또는 해당 라이선스에 명시된 대로 별도로 사용이 허가됩니다. 계속하면 이러한 라이선스에도 동의하는 것입니다.



비주얼 스튜디오 설치

Visual Studio Installer

설치됨 사용 가능



Visual Studio Community 2019

다운로드됨
100%

설치 중: 패키지 314/339
56%

Win10SDK_10.0.18362

☒ 설치 후 시작

[릴리스 정보](#)

일시 중지

개발자 뉴스

[Visual Studio 2019 version 16.5 is now available](#)

The Visual Studio 2019 team here in Redmond h...
2020년 3월 20일 금요일

[What's New for Xamarin Developers in Visual Studio 2019 version 16.5](#)

This week, Visual Studio 2019 version 16.5 was...
2020년 3월 20일 금요일

[Announcing .NET 5 Preview 1](#)

At the end of last year, we shipped .NET Core 3.0...
2020년 3월 20일 금요일

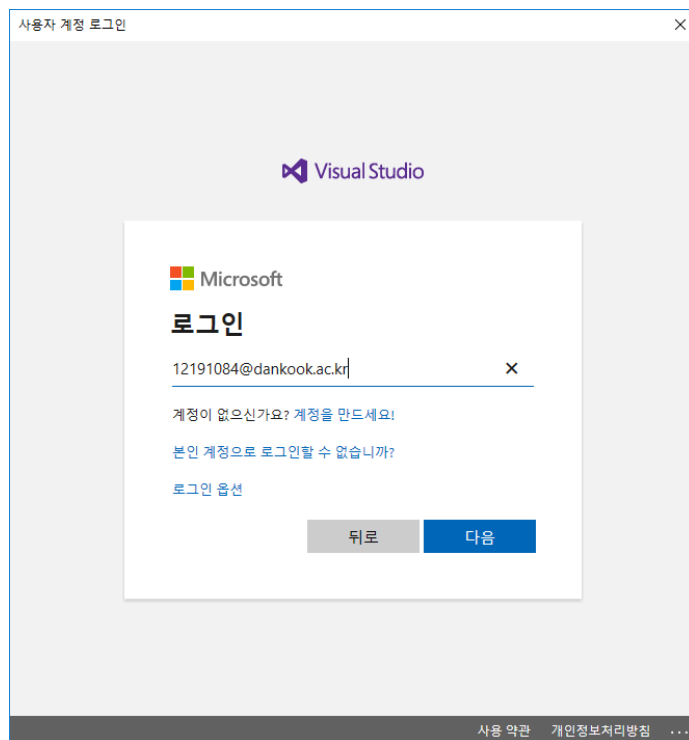
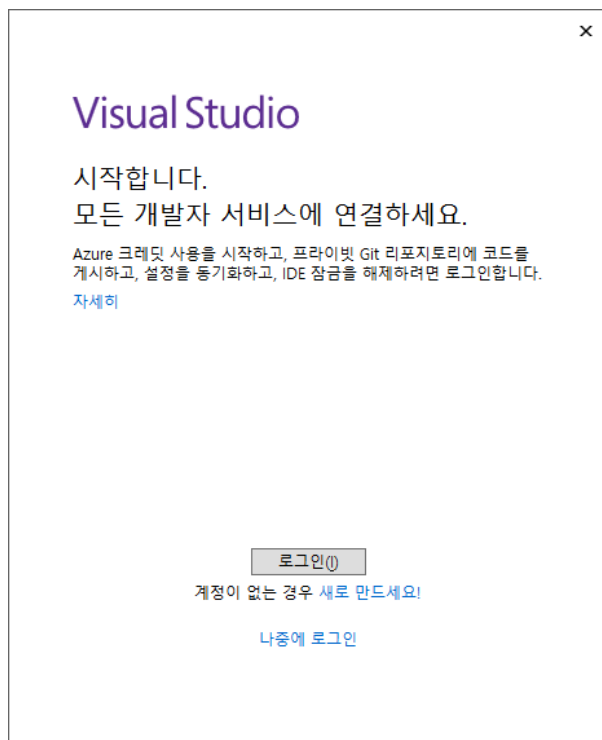
[온라인에서 뉴스 더 보기...](#)

도움이 필요하세요? [Microsoft Developer Community](#)를 확인하거나 [Visual Studio 지원](#)을 통해 문의하세요.

설치 관리자 버전 2.5.2057.204



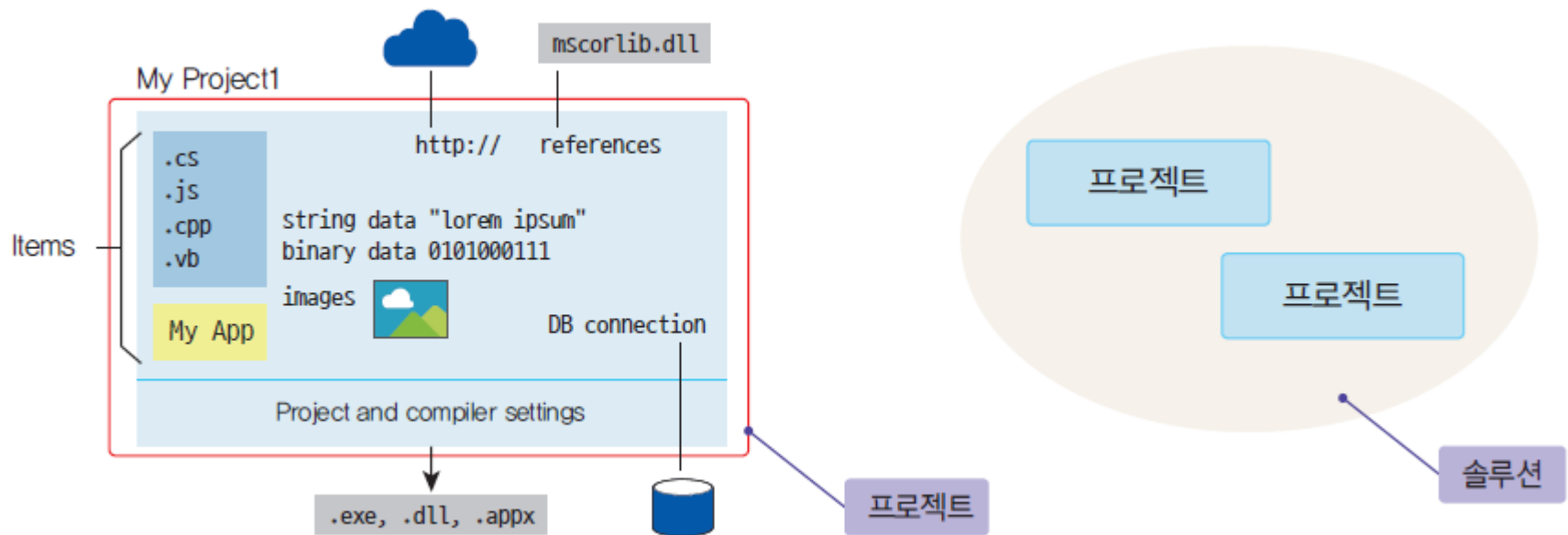
비주얼 스튜디오 설치





워크스페이스와 프로젝트

- **솔루션(solution)**; 문제 해결에 필요한 프로젝트가 들어 있는 컨테이너
- **프로젝트(project)**: 하나의 실행 파일을 만드는데 필요한 여러 가지 항목들이 들어 있는 컨테이너





프로젝트 생성하기

Visual Studio 2019

최근 파일 열기(R)

Visual Studio를 사용할 때 여는 프로젝트, 폴더 또는 파일은 빠른 액세스를 위해 여기에 표시됩니다.
항상 목록의 맨 위에 표시되도록 자주 여는 항목을 고정할 수 있습니다.

시작



코드 복제 또는 체크 아웃(C)

GitHub 또는 Azure DevOps 같은 온라인 리포지토리에서 코드 가져오기



프로젝트 또는 솔루션 열기(P)

로컬 Visual Studio 프로젝트 또는 .sln 파일 열기



로컬 폴더 열기(F)

폴더 내에서 탐색 및 코드 편집



새 프로젝트 만들기(N)

시작하려면 코드 스캐폴딩과 함께 프로젝트 템플릿을 선택하세요.

[코드를 사용하지 않고 계속\(W\) →](#)



프로젝트 생성하기

새 프로젝트 만들기

최근 프로젝트 템플릿(R)

📁 콘솔 앱

C++

🔍

템플릿 검색(Alt+S)(S)



모든 언어(L)

모든 플랫폼(P)

모든 프로젝트 형식(T)



빈 프로젝트

Windows용 C++를 사용하여 처음부터 시작합니다. 시작 파일을 제공하지 않습니다.

C++

Windows

콘솔



콘솔 앱

Windows 터미널에서 코드를 실행합니다. 기본적으로 "Hello World"를 출력합니다.

C++

Windows

콘솔



Windows 데스크톱 마법사

마법사를 사용하여 고유한 Windows 앱을 만드세요.

C++

Windows

데스크톱

콘솔

라이브러리



Windows 데스크톱 애플리케이션

Windows에서 실행되는 그래픽 사용자 인터페이스를 사용하는 애플리케이션용 프로젝트입니다.

C++

Windows

데스크톱



공유 항목 프로젝트

공유 항목 프로젝트는 여러 프로젝트 간에 파일을 공유하는 데 사용됩니다.

C++

Windows

Android

iOS

Linux

데스크톱

콘솔

라이브러리

UWP

게임

모바일

뒤로(B)

다음(N)



프로젝트 생성하기

×

Windows 데스크톱 프로젝트

애플리케이션 종류(T)

콘솔 애플리케이션(.exe) ▾

추가 옵션:

☒ 빈 프로젝트(E)

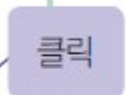
☐ 미리 컴파일된 헤더(P)

☐ 기호 내보내기(X)

☐ MFC 헤더(M)

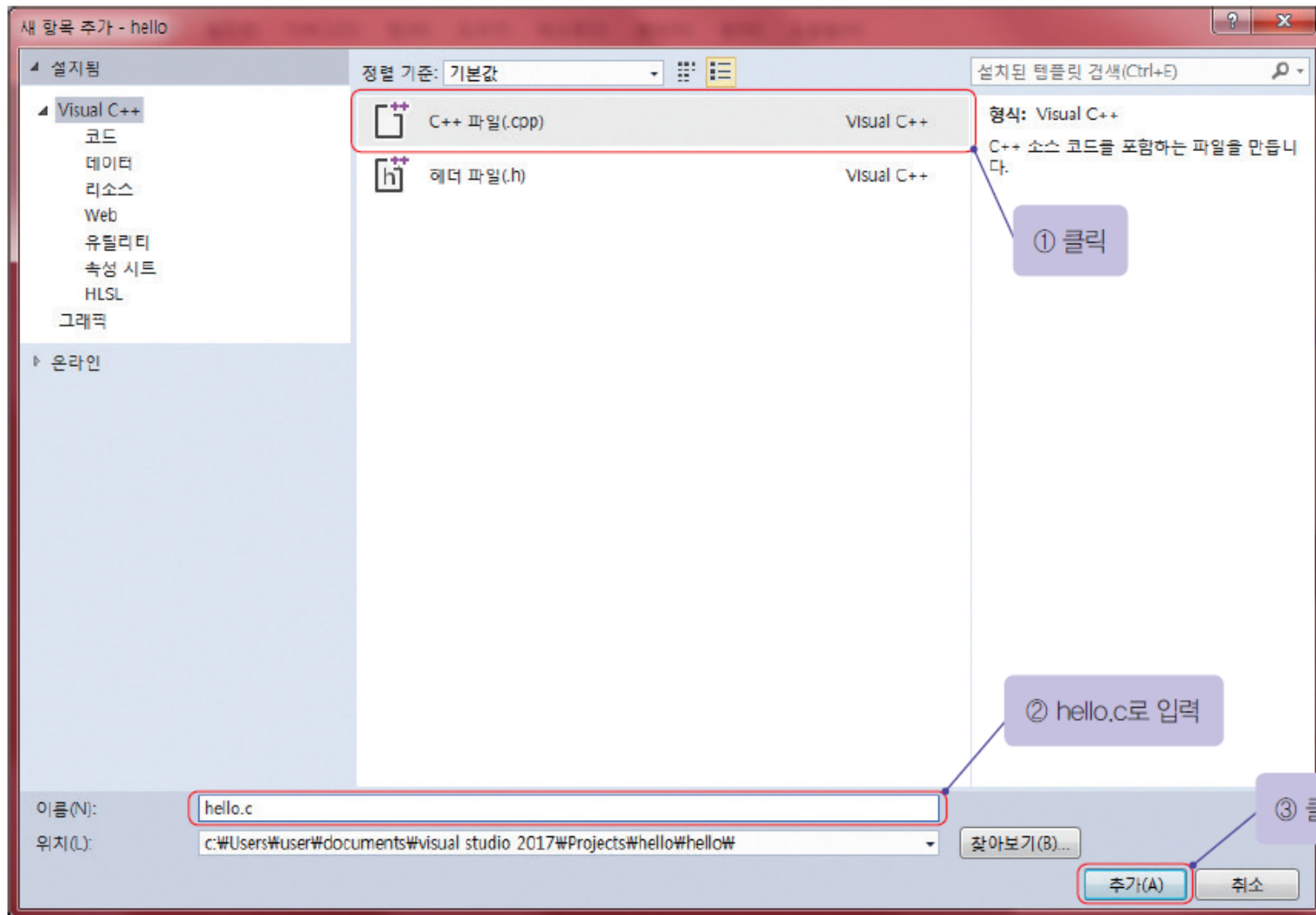
i 팁: 빈 프로젝트 템플릿을 사용하여 이 종류의 프로젝트를 만들 수도 있습니다.

확인취소





소스 파일 생성하기





프로그램 입력

The screenshot displays the Microsoft Visual Studio IDE with a C program named 'hello.c' open. The code is as follows:

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     printf("Hello World!");
6     return 0;
7 }
```

Annotations on the image include:

- A red box around the code is labeled "철자에 주의하여 입력한다." (Pay attention to spelling when entering).
- A purple box labeled "프로젝트" (Project) points to the 'hello' project in the Solution Explorer.
- A purple box labeled "솔루션" (Solution) points to the 'hello' solution in the Solution Explorer.

The Solution Explorer on the right shows the project structure:

- 솔루션 탐색기 (Solution Explorer)
- 솔루션 탐색기 검색 (Ctrl+Q)
- 솔루션 'hello' (1개 프로젝트)
- hello
 - 참조 (References)
 - 외부 종속성 (External Dependencies)
 - 리소스 파일 (Resource Files)
 - 소스 파일 (Source Files)
 - hello.c
 - 헤더 파일 (Header Files)

The bottom status bar shows the following information:

- 라인 6 (Line 6)
- 열 21 (Column 21)
- 문자 11 (Character 11)
- INS (Insert mode)
- 소스 제어에 추가 (Add to Source Control)



기호는 정확하게 입력

문장의 끝에는 세미콜론



첫 번째 프로그램의 설명

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    printf("Hello World!");
```

```
    return 0;
```

```
}
```





프로그램 == 작업 지시서

*화면에 "Hello World!"를
표시한다.

작업 지시서

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    printf("Hello World!");  
    return 0;
```

```
}
```

프로그램



작업을 적어주는 위치

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```



```
    return 0;
```

```
}
```

여기다 원하는 작업을
수행하는 문장을 적어준다.

프로그램



간략한 소스 설명

```
#include <stdio.h>
```

헤더파일을 포함한다.

```
int main(void)
```

메인 함수 시작

```
{
```

```
    printf("Hello World!");
```

화면에 "Hello World!"를 출력

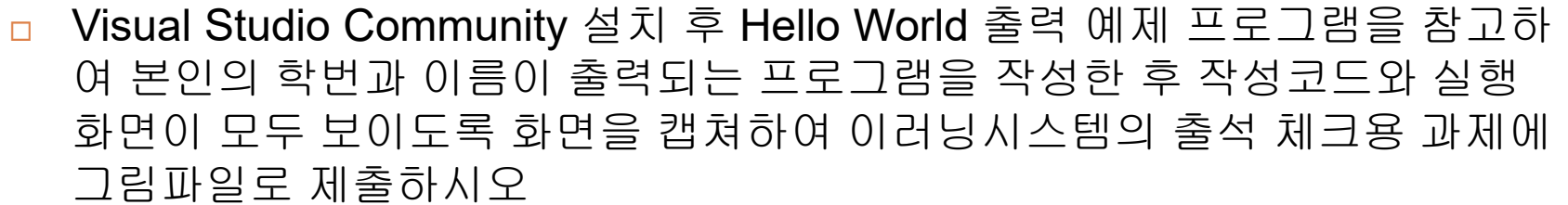
```
    return 0;
```

외부로 0값을 반환

```
}
```

메인 함수 종료

프로그램



나눔고덕 ▾ 10pt ▾ 가 간 거 과 감 **자** ▾ URL

사진 외부컨텐츠 수식입력 ☐ HTML



헤더 파일

- `#include`는 소스 코드 안에 특정 파일을 현재의 위치에 포함

- 주의!: 전처리기 지시자 문장 끝에는 세미콜론(;)을 붙이면 안 된다.

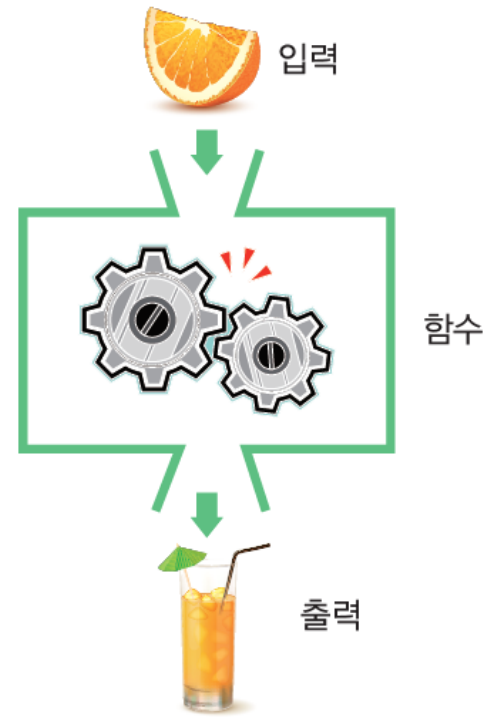
`#include <stdio.h>`

- 헤더 파일(header file): 컴파일러가 필요로 하는 정보를 가지고 있는 파일
- stdio.h: standard input output header file



함수

- 함수(function): 특정한 작업을 수행하기 위하여 작성된 독립적인 코드 $y = x^2 + 1$
- (참고) 수학적 함수
- 프로그램 = 함수의 집합





main() 함수

- 함수의 반환값

- 함수의 입력은 없음!

int main(void)

- 함수의 이름
- main()은 가장 먼저 수행되는 함수



함수의 간단한 설명

함수의 출력 타입

int

함수의 이름

main

함수의 입력 타입

(void)

{

함수의 시작

```
printf("Hello World");  
return 0;
```

함수의 몸체

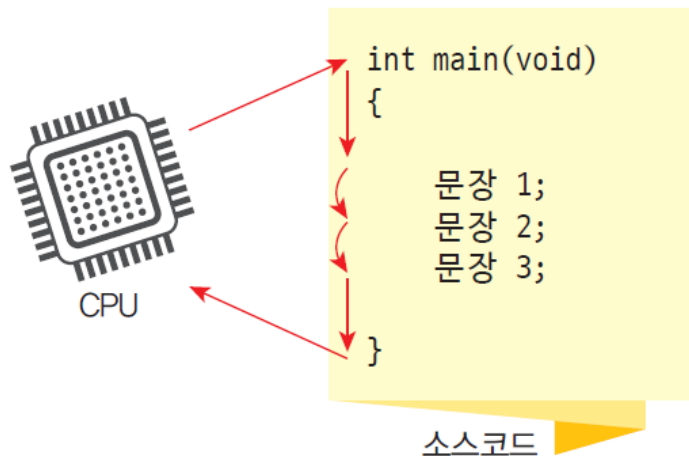
}

함수의 끝



문장

- 함수는 여러 개의 문장으로 이루어진다.
- 문장들은 순차적으로 실행된다.
- 문장의 끝에는 반드시 ;이 있어야 한다.



소스 코드의 문장들은
기본적으로 차례대로
실행됩니다.





printf() 호출

- printf()는 컴파일러가 제공하는 함수로서 출력을 담당한다

printf("Hello World!");

- 큰따옴표 안의 문자열이 화면에 출력된다.





함수의 반환값

- return은 함수의 결과값을 외부로 반환

return 0;

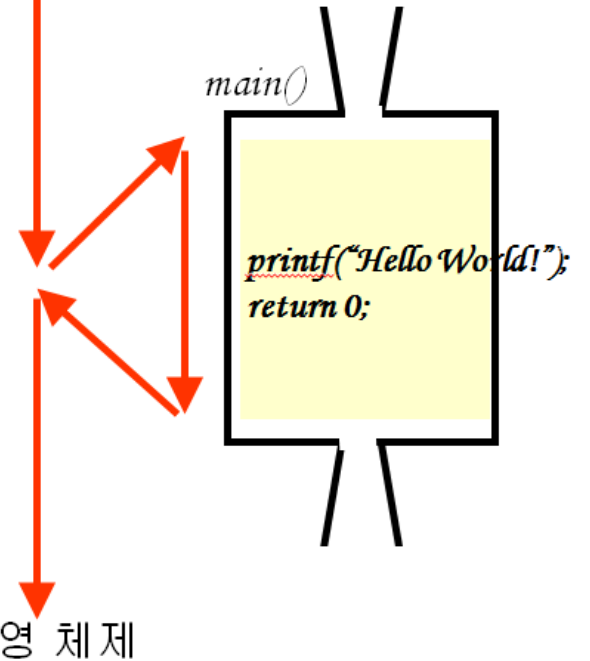
- 반환값은 0

운영 체제

main()

```
printf("Hello World!");  
return 0;
```

운영 체제





응 프로그램 #1

- 다음과 같은 출력을 가지는 프로그램을 제작하여 보자.





첫 번째 버전

- 문장들은 순차적으로 실행된다는 사실 이용

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    printf("Hello World!");
```

```
    printf("Kim ChulSoo");
```

```
    return 0;
```

```
}
```

- 2개의 문장은
순차적으로 실행된다



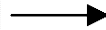
Hello World! Kim ChulSoo



줄바꿈 문자

- 줄바꿈 문자인 `\n`은 화면에서 커서는 다음줄로 이동하게 한다.

```
printf("Hello World!");
```



현재 커서의 위치.
다음 문자를 표시할 때는
이곳부터 시작한다.

```
printf("Hello World!\n");
```





잘바꿈 문자 2개를 사용하면?

```
printf("Hello \nWorld! \n");
```





변경된 프로그램

- 줄바꿈 문자를 포함하면 우리가 원하던 결과가 된다.

```
#include <stdio.h>

int main(void)
{

    printf("Hello World!\n");
    printf("Kim ChulSoo \n");

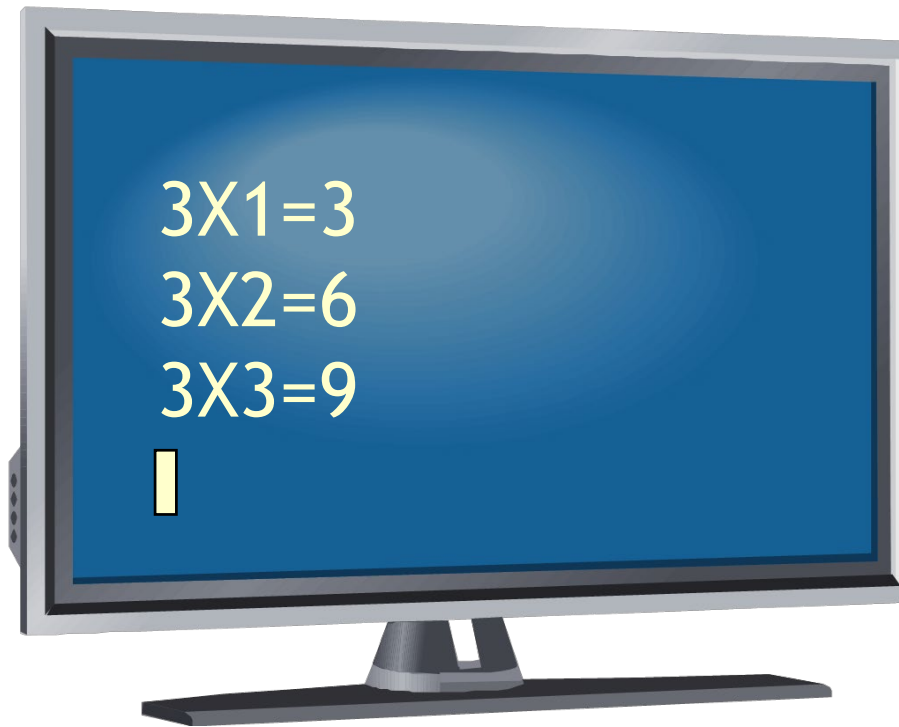
    return 0;
}
```





오늘 프로그램 #2

- 다음과 같은 출력을 가지는 프로그램을 제작하여 보자.





- 역시 문장들은 순차적으로 수행된다는 점을 이용한다.

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    printf("3 X 1 = 3\n");
```

```
    printf("3 X 2 = 6\n");
```

```
    printf("3 X 3 = 9\n");
```

```
    return 0;
```

```
}
```

3개의 문장은
순차적으로 실행된다.



lab: 간단한 계산을 해보자

- 덧셈과 뺄셈, 곱셈, 나눗셈 계산을 하는 프로그램을 작성해보자.

📌 실행결과

$2+5=5$

$2-3=-1$

$2*3=6$

$2/3=0$



solution

```
#include <stdio.h>

int main(void)
{
    printf("2+5=%d\n", 2 + 3);
    printf("2-3=%d\n", 2 - 3);
    printf("2*3=%d\n", 2 * 3);
    printf("2/3=%d\n", 2 / 3);
    return 0;
}
```



형식 지정자

- 형식 지정자: `printf()`에서 값을 출력하는 형식을 지정한다.

형식 지정자	의미	예	실행 결과
%d	10진 정수로 출력	<code>printf("%d \n", 10);</code>	10
%f	실수로 출력	<code>printf("%f \n", 3.14);</code>	3.14
%c	문자로 출력	<code>printf("%c \n", 'a');</code>	a
%s	문자열로 출력	<code>printf("%s \n", "Hello");</code>	Hello



오류 수정 및 디버깅

- 컴파일이나 실행 시에 오류가 발생할 수 있다.
- 에러와 경고
 - ▣ 에러(error): 심각한 오류
 - ▣ 경고(warning): 경미한 오류

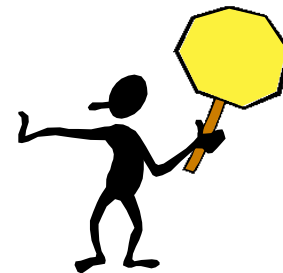




오류의 종류

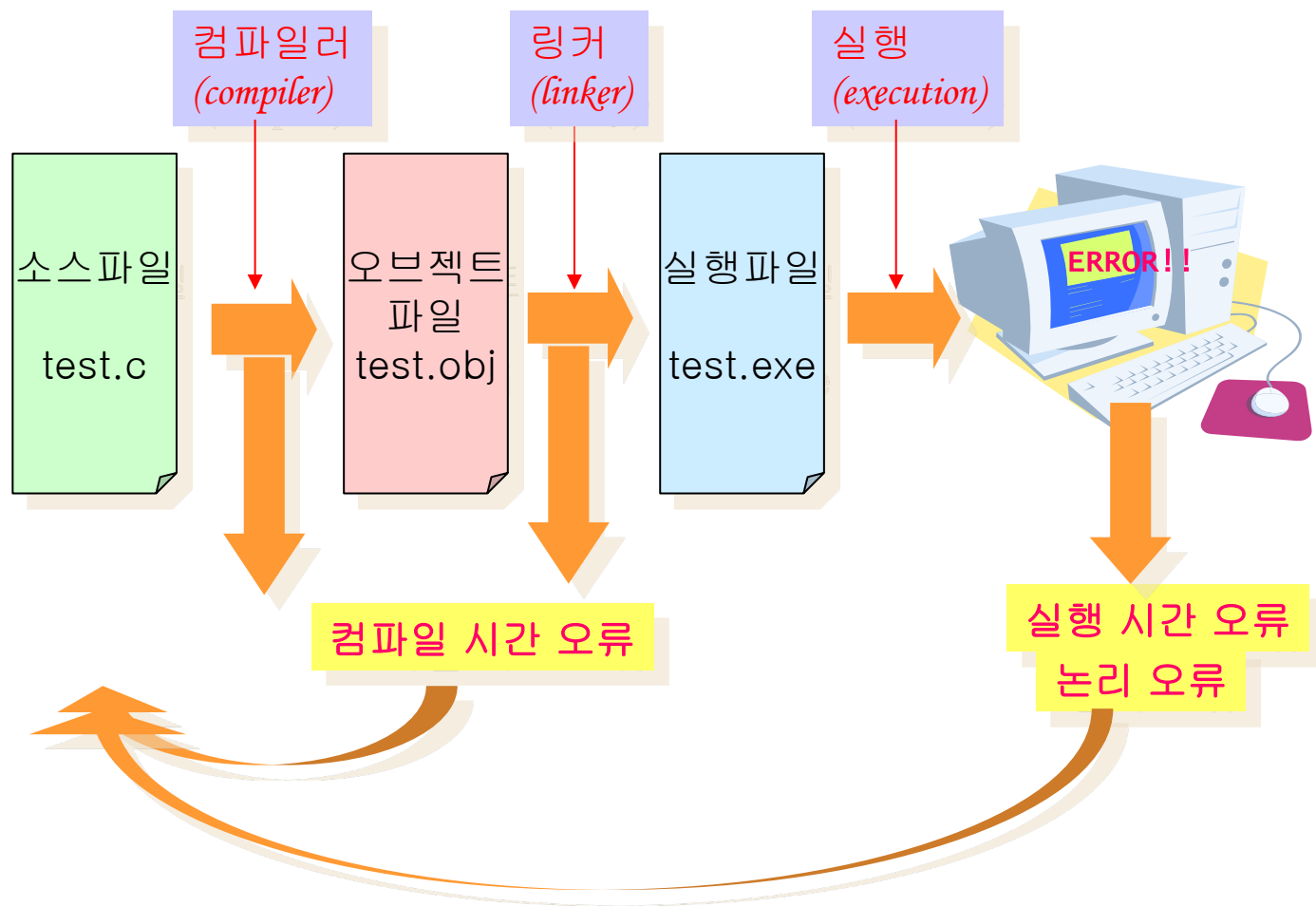
□ 오류의 종류

- ▣ 컴파일 시간 오류: 대부분 문법적인 오류
- ▣ 실행 시간 오류: 실행 중에 0으로 나누는 연산 같은 오류
- ▣ 논리 오류: 논리적으로 잘못되어서 결과가 의도했던 대로 나오지 않는 오류





오류 수정 과정





오류 #1

add - Microsoft Visual Studio

빠른 실행 (Ctrl+Q)

파일(F) 편집(E) 보기(V) 프로젝트(P) 빌드(B) 디버거(D) 팀(M) 도구(T) 테스트(S) 분석(N) 창(W) 도움말(H) 로그인

Debug x86 로컬 Windows 디버거

error1.c

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     printf("Hello World!");
6     return 0;
7 }
```

;이 생략되었다.

retrun 앞에 ;를 빠뜨렸다는 의미이다.

오류가 발견된 소스 파일

오류가 발견된 줄번호

오류 목록

전체 솔루션 2 오류 0 경고 0 메시지 빌드 + IntelliSense 검색 오류 목록

코드	설명	프로젝트	파일	줄	비표시 오류(Suppr...)
C2143	구문 오류: ';'이(가) 'return' 앞에 없습니다.	add	error1.c	6	
		add	error1.c	6	

오류 목록 출력

준비 줄: 7 열: 2 문자: 2 INS

↑ 게시

솔루션 탐색기

솔루션 탐색기 검색 (Ctrl+,)

솔루션 'add' (1개 프로젝트)

- add
 - 참조
 - 외부 종속성
 - 리소스 파일
 - 소스 파일
 - error1.c
 - 헤더 파일

속성

main VCppCodeFunction

C++	
(Name)	main
File	d:\sources\Wch
FullName	main



오류 #2

add - Microsoft Visual Studio

빠른 실행(Ctrl+Q)

파일(F) 편집(E) 보기(V) 프로젝트(P) 빌드(B) 디버그(D) 팀(M) 도구(T) 테스트(S) 분석(N) 창(W) 도움말(H) 로그인

Debug x86 로컬 Windows 디버거

error3.c

add (전역 범위) main(void)

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     printf("Hello World!");
6     return 0;
7 }
```

문자열을 표시할 때, "와"를 빠뜨렸다.

오류 목록

전체 솔루션 5 오류 2 경고 0 메시지 빌드 + IntelliSense 검색 오류 목록

코드	설명	프로젝트	파일	줄	비표시 오류(Suppr...)
abc	식별자 "Hello"이(가) 정의되어 있지 않습니다.	add	error3.c	5	
abc	')'가 필요합니다.	add	error3.c	5	
C2065	'Hello': 선언되지 않은 식별자입니다.	add	error3.c	5	
C4047	'함수': 'const char *const'의 간접 참조 수준이 'int'과(와) 다릅니다.	add	error3.c	5	

오류 목록 출력

속성

main VCodeFunction

C++

(Name)	main
File	d:\sources\Wch
FullName	main

C++

준비 줄: 6 열: 14 문자: 11 INS 게시



오류 #3

add - Microsoft Visual Studio

빠른 실행(Ctrl+Q)

파일(F) 편집(E) 보기(V) 프로젝트(P) 빌드(B) 디버그(D) 팀(M) 도구(T) 테스트(S) 분석(N) 창(W) 도움말(H) 로그인

Debug x86 로컬 Windows 디버거

error3.c

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     print("Hello World!");
6     return 0;
7 }
```

printf이어야 한다.

오류 목록

전체 솔루션 2 오류 1 경고 0 메시지 빌드 + IntelliSense

코드	설명	프로젝트	파일	줄	미표시 오류(Suppr...)
C4013	'print'이(가) 정의되지 않았습니다. extern은 int형을 반환하는 add 것으로 간주합니다.		error3.c	5	
LNK2015	_print 외부 기호(참조 위치: _main 함수)에서 확인하지 못했습니다.		error3.obj	1	
LNK1121	1개의 확인할 수 없는 외부 참조입니다.	add	add.exe	1	

오류 목록 출력

함수를 찾지 못했음.

솔루션 탐색기

솔루션 탐색기 검색(Ctrl+;)

솔루션 'add' (1개 프로젝트)

- add
 - 참조
 - 외부 종속성
 - 리소스 파일
 - 소스 파일
 - error3.c
 - 헤더 파일

속성

main VCodeFunction

C++

(Name)	main
File	d:\sources\Wch
FullName	main

C++

빌드 실패

↑ 게시



- 다음과 같은 출력을 가지는 프로그램을 작성하여 보자.





논리 오류가 존재하는 프로그램

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    printf("Hey!");
```

```
    printf("Good Morning");
```

```
    return 0;
```

```
}
```

줄이 바뀌지
않았음!

Hey!Good Morning





논리 오류가 수정된 프로그램

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    printf("Hey! \n");
```

```
    printf("Good Morning \n");
```

```
    return 0;
```

```
}
```

논리 오류
수정!!



Hey!
Good Morning



디버깅

□ 디버깅: 논리 오류를 찾는 과정

아무래도 이부분이 수상해..



프로그램의 실행결과

논리 에러를 발견하는 것은
수사관이 범죄 흔적을 이용하여
범인을 찾는 것과 같습니다.





디버거(debugger)

hello (디버깅) - Microsoft Visual Studio

파일(F) 편집(E) 보기(V) 프로젝트(P) 빌드(B) 디버그(D) 팀(M) 도구(T) 테스트(S) 분석(N) 창(W) 도움말(H)

프로세스: [144720] hello.exe

hello.c

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     printf("Hello World!");
6     printf("Good Mornin");
7     return 0;
8 }
```

현재 실행되고 있는 위치

디버그(D) 메뉴:

- 장(W)
- 그래픽(C)
- 계속(C) F5
- 모두 중단(K) Ctrl+Alt+Break
- 디버깅 중지(E) Shift+F5
- 모두 분리(L)
- 모두 종료(M)
- 다시 시작(R) Ctrl+Shift+F5
- 코드 변경 내용 적용(A) Alt+F10
- 성능 프로파일러(F...) Alt+F2
- 프로세스에 연결(P...) Ctrl+Alt+P
- 기타 디버그 대상(H)
- 프로파일러
- 하 단계씩 코드 실행(I) F11
- 프로세서 단위 실행(O) F10
- 프로세서 나가기(T) Shift+F11
- 간략한 조사식(Q...) Shift+F9
- 중단점 설정/해제(G) F9
- 새 중단점(B)
- 모든 중단점 삭제(A) Ctrl+Shift+F9
- 모든 DataTips 지우기(A)
- DataTips 내보내기(X)...
- DataTips 가져오기(I)...
- 다른 이름으로 덤프 저장(V)...
- 옵션(O)...
- hello 속성...

빠른 실행(Ctrl+Q)

진단 도구

진단 및 세션: 0 초(178 ms이(가) 선택됨)

CPU (모든 프로세서에 대한 비율(%))

요약 이벤트 메모리 사용량 CPU 사용량

이벤트

표시(1/1)

스냅숏 만들기

한 문장 단위로 실행한다.

준비

↑ 소스 제어에 추가



디버거의 명령어 정의

- F5 (Go): 실행
- F10 (Step Over): 한 문장씩 실행(함수도 하나의 문장 취급)
- F11 (Step Into): 한 문장씩 실행(함수 안으로 진입)
- F9 (Breakpoint): 현재 문장에 중단점을 설정



디버거의 실행 과정

F10을 누를 때마다
한 문장씩 실행된다.

F10을 누를 때마다
한 문장씩 실행된다.

The image shows three sequential screenshots of the Visual Studio IDE, illustrating the step-through debugging process for a C program named `hello.c`. The code is as follows:

```
(전역 범위)
- main(void)
{
    /* 첫 번째 프로그램 */
    #include <stdio.h>

    int main(void)
    {
        printf("Hello World!\n");
        printf("Good Morning\n");
        return 0;
    }
}
```

Top Screenshot: The debugger is at the first line of the `main` function. The console window shows "Hello World!".

Middle Screenshot: The debugger has moved to the second line of the `main` function. The console window shows "Hello World!" and "Good Morning".

Bottom Screenshot: The debugger has moved to the third line of the `main` function. The console window shows "Hello World!" and "Good Morning".



mini project

□ 오류를 수정해보자!

```
#include <stdio.h>

int Main(void)
(
    printf(안녕하세요?\n);
    printf(이번 코드에는 많은 오류가 있다네요\n);
    print(제가 다 고쳐보겠습니다.\n);
    return 0;
)
```




bug.c

```
1  #include <stdio.h>
```

```
2
```

main

```
3  int Main(void)
```

(가 아니라 {이어야 한다.

```
4  (
```

```
5      printf(안녕하세요? \n);
```

```
6      printf(이번 코드에는 많은 오류가 있다네요 \n)
```

문장의 끝에는 ;가 있어야 한다.

```
7      print(제가 다 고쳐보겠습니다.\n);
```

```
8      return 0;
```

print가 아니고 printf이어야 한다.

문자열에는 따옴표를 붙인다.

```
9  )
```



Mini Project

□ 오류를 수정해보자!

```
#include <stdio.h>

int main(void)
{
    printf("안녕하세요 ? \n");
    printf("이번 코드에는 많은 오류가 있다네요\n");
    printf("제가 다 고쳐보겠습니다.\n");
    return 0;
}
```



출석 체크용 질문



- 아래의 질문에 대한 답변을 이러닝 시스템의 과제 제출란에 작성 하시오.
(수업 시작시간에서 1시간 30분 이내 제출, 이후 지각처리)
- 1) 프로그램이 실행될 때 가장 먼저 실행되는 함수는 무엇인지 쓰시오.
- 2) 함수의 구조에 대해 설명 하시오.



Q & A

